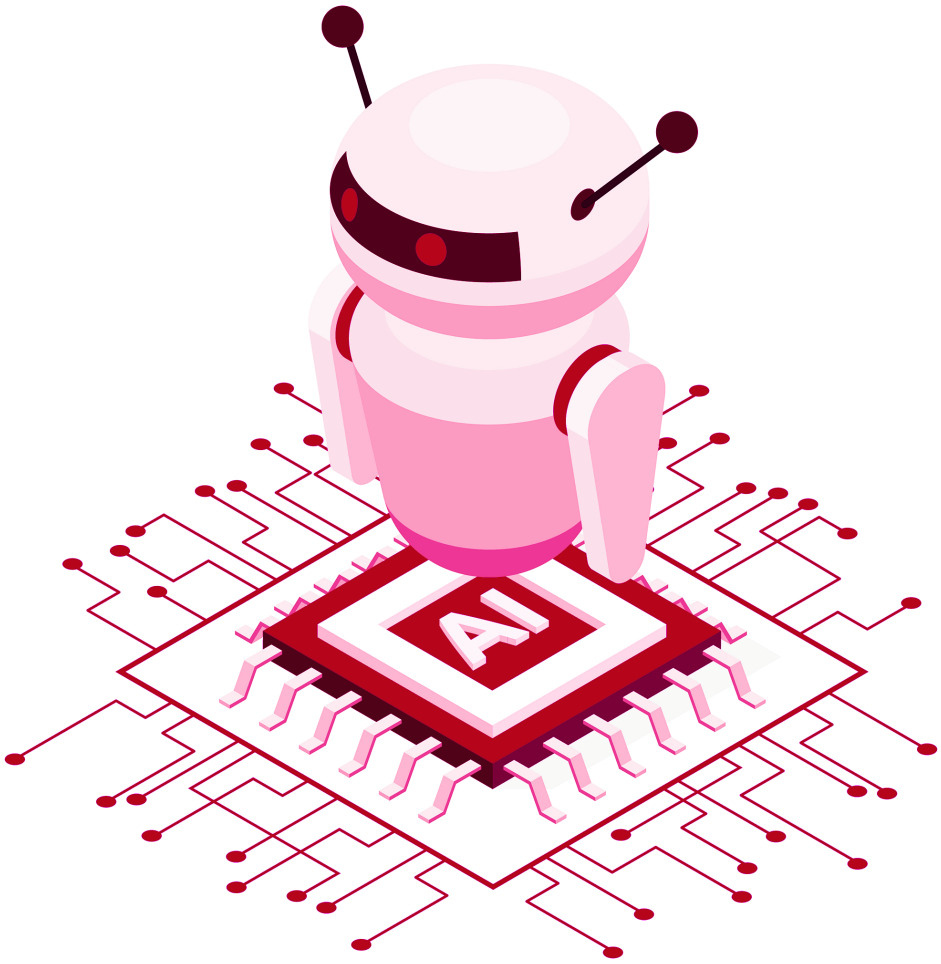

Deep Reinforcement Learning

Professor Mohammad Hossein Rohban

Homework 6:

Multi-Armed Bandits

By:

Parsa Ghezelbash

401110437

RIML

Spring 2025

Contents

Grading

The grading will be based on the following criteria, with a total of 105.25 points:

Task	Points
Task 1: Oracle Agent	3.5
Task 2: Random Agent	2
Task 3: Explore-First Agent	5.75
Task 4: UCB Agent	7
Task 5: Epsilon-Greedy Agent	2.25
Task 6: LinUCB Agent	27.5
Task 7: Final Comparison and Analysis	6
Task 8: Final Deep-Dive Questions	41.25
Clarity and Quality of Code	5
Clarity and Quality of Report	5
Bonus 1: Writing your report in Latex	10

0 Task 0: MAB Overview

- Questions:
  - How might the performance of different agents change if the distribution of probabilities were not uniform?
  - Why does the MAB environment use a simple binary reward mechanism (1 or 0)?

1 Task 1: Oracle Agent

- The Oracle uses privileged information to determine the maximum expected reward.
- **TODO:** Compute the oracle reward (2 points).
- Questions:
  - What insight does the oracle reward provide? (0.75 points)
    Answer:
    It says that the maximum reward per step can't exceed 0.9636627605010293.
  - Why is the oracle considered cheating? (0.75 points)
    Answer:
    Because it knows the distribution of probabilities and uses it to do the best action.

2 Task 2: Random Agent (RndAg)

- This agent selects actions uniformly at random.
- **TODO:** Choose a random action (1 point).
- Questions:
  - Why is its reward lower and highly variable? (0.25 points)
  - How could the agent be improved without learning? (0.75 points)

3 Task 3: Explore-First Agent (ExpFstAg)

- The agent explores randomly for a fixed number of steps and then exploits the best arm.
- **TODOs:**
  - Update Q-value (3 points)
  - Choose action based on exploration versus exploitation (1 point)
- Questions:
  - Why might the early exploration phase (e.g., 5 steps) lead to high fluctuations in the reward curve? (0.75 points)
    Answer:
    Because the agent collects only a small number of samples for each arm, the estimates of expected rewards are highly noisy. As a result, the agent may prematurely commit to a suboptimal arm, causing large fluctuations in performance when exploitation begins.
  - What are the trade-offs of using a fixed exploration phase? (1 point)
    Answer:
    A short exploration phase allows quicker exploitation, but increases the risk of selecting a suboptimal arm. A longer exploration phase improves accuracy in estimating arm values but

delays exploitation and may reduce cumulative reward in the short term.

- How does increasing max_ex affect the convergence of the agents performance?
  Answer:
  Increasing `max_ex` allows the agent to gather more accurate reward estimates, which improves the likelihood of selecting the optimal arm. This can enhance long-term performance and stability but delays convergence since exploitation starts later.
- In real-world scenarios, what challenges might arise in selecting the optimal exploration duration?
  Answer:
  Real-world environments are often non-stationary or partially observable, making it hard to determine when exploration should stop.

4 Task 4: UCB Agent (UCB_Ag)

- Uses an exploration bonus to balance learning.
- **TODOs:**
  - Update Q-value (3 points)
  - Compute the exploration bonus (4 points)

5 Task 5: UCB vs ExpFstAg

- Questions:
  - Why does UCB learn slowly (even after 500 steps, not reaching maximum reward)? *Hint:* Consider the conservative nature of the exploration bonus.

    Answer:

    UCB is designed to balance exploration and exploitation based on uncertainty. Early on, the exploration bonus is large, encouraging broad sampling. However, UCB remains cautious by continuing to allocate some probability to underexplored arms even when a good arm has already been identified. This conservative exploration slows convergence, especially in finite-time scenarios.

  - Under what conditions might an explore-first strategy outperform UCB, despite UCBs theoretical optimality?

    Answer:

    If the optimal arm can be identified reliably with a small number of exploratory samples, the Explore-First Agent (ExpFstAg) may quickly lock onto it and outperform UCB in terms of cumulative reward. This often happens in environments where the differences between arm rewards are large and noise is low.

  - How do the design choices of each algorithm affect their performance in short-term versus long-term scenarios?

    Answer:

    ExpFstAg frontloads exploration, allowing for rapid exploitation which benefits short-term reward. However, it is rigid and cannot adapt if initial estimates are inaccurate. UCB, on

the other hand, continues to explore, which may reduce short-term reward but leads to better long-term performance.

- What happens if we let ExpFstAg explore for 20 steps? Compare its reward to UCB.

  Answer:

  With 20 exploration steps, ExpFstAg has more opportunity to estimate the reward of each arm accurately. The additional data improves arm selection, the agent outperforms UCB in cumulative reward from early stages.

- What impact does increasing the exploration phase to 20 steps have compared to 5 steps?

  Answer:

  Increasing the exploration phase generally reduces the variance in reward estimates, making it more likely the agent picks the best arm. While this delays exploitation.

- How can you determine the optimal balance between exploration and exploitation in practice?

  Answer:

  The optimal balance is typically determined empirically through cross-validation or hyperparameter tuning.

- We know that UCB is optimal. Why might ExpFstAg perform better in practice? (Discuss how hyperparameter tuning and early exploitation can sometimes yield higher rewards in finite-time scenarios.)

  Answer:

  UCB's optimality is asymptotic, meaning it guarantees low regret in the long run. In practice, with limited time or episodes, a well-tuned ExpFstAg can exploit early gains by quickly settling on a good arm. If the exploration phase is sufficient, it can outperform UCB due to faster exploitation and reduced conservative sampling.

6 Task 6: Epsilon-Greedy Agent (EpsGdAg)

- Selects the best-known action with probability $1 - \varepsilon$ and a random action with probability ε.
- **TODO:** Choose a random action based on ε (1 point)
- Questions:
  - Why does a high ε result in lower immediate rewards? (0.5 points)
    Answer:
    A high ε causes the agent to select random actions more frequently, which increases the likelihood of choosing suboptimal arms. As a result, the immediate rewards are lower because the agent does not consistently exploit the best-known arm.
  - What benefits might decaying ε over time offer? (0.75 points)
    Answer:
    Decaying ε allows the agent to explore more during the early stages and gradually shift toward exploitation. This balances exploration and exploitation more effectively, improving long-term reward.
  - How do the reward curves for different ε values reflect the explorationexploitation balance? (1.25 points)

Answer:
Higher ε values lead to flatter reward curves initially due to more exploration and noise. Lower ε curves tend to rise more quickly if the agent exploits a good arm early, but may plateau at suboptimal levels due to insufficient exploration like $\varepsilon = 0.4, 0.2$. the best reward was reached by $\varepsilon = 0.1$.

- Under what circumstances might you choose a higher ε despite lower average reward? (1.25 points)
  Answer:
  In environments where arm rewards are close together or highly non-stationary, a higher ε ensures continued exploration and adaptability.

7 Task 7: LinUCB Agent (Contextual Bandits)

- Leverages contextual features using a linear model.
- **TODOs:**
  - Compute UCB for an arm given context (7 points)
  - Update parameters A and b for an arm (7 points)
  - Compute UCB estimates for all arms (7 points)
  - Choose the arm with the highest UCB (4 points)
- Questions:
  - How does LinUCB leverage context to outperform classical bandit algorithms? (1.25 points)
    Answer:
    LinUCB uses contextual features to make arm selection decisions based on the current state, rather than relying solely on average rewards. By learning a separate linear model for each arm, it can generalize from past experiences to similar contexts.
  - What is the role of the α parameter in LinUCB, and how does it affect the exploration bonus? (1.25 points)
    Answer:
    The α parameter controls the trade-off between exploration and exploitation. It scales the uncertainty term in the UCB estimate. A larger α encourages more exploration, while a smaller α leads to more greedy behavior focused on estimated rewards.
  - What does α affect in LinUCB? (1.25 points)
    Answer:
    α directly affects how aggressively the agent explores uncertain arms. It determines the magnitude of the confidence interval added to the expected reward when calculating the UCB.
  - Do the reward curves change with α? Explain why or why not. (1.25 points)
    Answer:
    Yes, reward curves change with α. A small α leads to faster initial gains (like $\alpha = 0.01, 0.1$ but may get stuck in local optima, while a large α results in lower short-term rewards due to heavy exploration, but potentially better long-term performance if continued.
  - Based on your experiments, does LinUCB outperform the standard UCB algorithm? Why or why not? (1.25 points)
    Answer:
    Yes, In my experiment LinUCB reached a better reward with less regret and smoother reward curve. Because it uses richer contextual information to guide decisions.
  - What are the key limitations of each algorithm, and how would you choose between them for a given application? (1.25 points)

Answer:
UCB is simple and effective in low-dimensional and context-free environments, but cannot leverage rich features. LinUCB is more powerful in contextual settings but can be computationally expensive. If the application involves rich user or environmental context, LinUCB is preferred. In simpler or highly noisy settings with small feature dimension, classic UCB is preferred.

8 Task 8: Final Deep-Dive Questions

- **Finite-Horizon Regret and Asymptotic Guarantees** (4 points)

  Many algorithms (e.g., UCB) are analyzed using asymptotic (longterm) regret bounds. In a finite-horizon scenario (say, 5001000 steps), explain intuitively why an algorithm that is asymptotically optimal may still yield poor performance. What tradeoffs arise between aggressive early exploration and cautious longterm learning? Deep Dive: Discuss how the exploration bonus, tuned for asymptotic behavior, might delay exploitation in finite time, leading to high early regret despite eventual convergence.

  Answer:
  Asymptotically optimal algorithms like UCB are designed to minimize regret over an infinite time horizon. However, in finite-horizon scenarios, the exploration bonus may cause the algorithm to continue trying suboptimal arms longer than necessary. This delays exploitation of the best arm and increases short-term regret. The trade-off is between aggressive early exploration (to ensure long-term learning) and immediate performance. A high exploration bonus tuned for asymptotic behavior might result in overly cautious exploitation in the early phases.
- **Hyperparameter Sensitivity and ExplorationExploitation Balance** (4.5 points)

  Consider the impact of hyperparameters such as ϵ in ϵgreedy, the exploration constant in UCB, and the α parameter in LinUCB. Explain intuitively how slight mismatches in these parameters can lead to either underexploration (missing the best arm) or overexploration (wasting pulls on suboptimal arms). How would you design a selfadaptive mechanism to balance this tradeoff in practice? Deep Dive: Provide insight into the fragility of these parameters in finite runs and how a metaalgorithm might monitor performance indicators (e.g., variance in rewards) to adjust its exploration dynamically.

  Answer:
  Hyperparameters like ϵ in ϵ-greedy, the exploration constant in UCB, and α in LinUCB critically affect the balance between exploration and exploitation. Slight mismatches can lead to under-exploration (missing the optimal arm) or over-exploration (wasting time on bad arms). In finite runs, these parameters are fragile, small errors in tuning can severely hurt performance. A self-adaptive mechanism could monitor reward variance, convergence of Q-values, or rate of change in estimated rewards, and adjust exploration dynamically, for example decaying ϵ or adjusting α based on stability.
- **Context Incorporation and Overfitting in LinUCB** (4 points)

  LinUCB uses context features to estimate arm rewards, assuming a linear relation. Intuitively, why might this linear assumption hurt performance when the true relationship is complex or when the context is highdimensional and noisy? Under what conditions can adding context lead to worse performance than classical (contextfree) UCB? Deep Dive: Discuss the risk of overfitting

to noisy or irrelevant features, the curse of dimensionality, and possible mitigation strategies (e.g., dimensionality reduction or regularization).

Answer:
LinUCB assumes a linear relationship between context and reward, which can hurt performance if the true relation is non-linear or if the context is high-dimensional and noisy. In such cases, LinUCB might overfit and fail to generalize. Contextual information can reduce performance if it adds noise or irrelevant features. The curse of dimensionality increases estimation variance, especially with limited data. Mitigation strategies include feature selection, regularization, or using dimensionality reduction techniques like PCA.

- **Adaptive Strategy Selection** (4.25 points)

  Imagine designing a hybrid bandit agent that can switch between an explorefirst strategy and UCB based on observed performance. What signals (e.g., variance of reward estimates, stabilization of Qvalues, or sudden drops in reward) might indicate that a switch is warranted? Provide an intuitive justification for how and why such a metastrategy might outperform either strategy alone in a finitetime setting. Deep Dive: Explain the challenges in detecting when exploration is enough and how early exploitation might capture transient improvements even if the longterm guarantee favors UCB.

  Answer:
  A hybrid strategy could monitor metrics like reward variance, Q-value stabilization, or reward drops to decide when to switch from explore-first to UCB. For instance, if the variance of rewards or changes in Q-values fall below a threshold, it might indicate sufficient exploration. Early exploitation might capture transient rewards that UCB would delay discovering. Such a meta-strategy could outperform either algorithm by being more responsive to observed data.

- **Non-Stationarity and Forgetting Mechanisms** (4 points)

  In nonstationary environments where reward probabilities drift or change abruptly, standard bandit algorithms struggle because they assume stationarity. Intuitively, explain how and why a forgetting or discounting mechanism might improve performance. What challenges arise in choosing the right decay rate, and how might it interact with the exploration bonus? Deep Dive: Describe the delicate balance between retaining useful historical information and quickly adapting to new trends, and the potential for chasing noise if the decay is too aggressive.

  Answer:
  In non-stationary environments, past data may become outdated. Forgetting mechanisms, such as exponential decay or sliding windows, prioritize recent information. This improves adaptability, but the decay rate must be chosen carefully. Too aggressive decay leads to chasing noise, while too slow decay fails to adapt to changes. A well-tuned mechanism balances retaining useful data with being responsive to change.

- **Exploration Bonus Calibration in UCB** (3.75 points)

  The UCB algorithm adds a bonus term that decreases with the number of times an arm is pulled. Intuitively, why might a conservative (i.e., high) bonus slow down learningeven if it guarantees asymptotic optimality? Under what circumstances might a less conservative bonus be beneficial, and what risks does it carry? Deep Dive: Analyze how a high bonus may force the algorithm to continue sampling even when an arms estimated reward is clearly suboptimal, thereby delaying convergence. Conversely, discuss the risk of prematurely discarding an arm if the bonus is too low.

Answer:
A conservative (high) exploration bonus causes the UCB agent to over-sample uncertain arms, even if they appear suboptimal, which slows down learning. In contrast, a lower bonus may speed up convergence but risks prematurely discarding potentially good arms due to variance. In practice, tuning the bonus is crucial to balance exploration with convergence speed.

- **Exploration Phase Duration in Explore-First Strategies** (4 points)

  In the ExploreFirst agent (ExpFstAg), how does the choice of a fixed exploration period (e.g., 5 vs. 20 steps) affect the regret and performance variability? Provide a scenario in which a short exploration phase might yield unexpectedly high regret, and another scenario where a longer phase might delay exploitation unnecessarily. Deep Dive: Discuss how the optimal exploration duration can depend heavily on the underlying reward distributions variance and the gap between the best and other arms, and why a onesizefitsall approach may not work in practice.

  Answer:
  A short exploration phase may yield high regret if the best arm is under-sampled early, especially when arm means are close or noisy. A longer phase provides better estimates but delays exploitation. The optimal duration depends on the variance in rewards and the expected gap between the best and other arms. A fixed phase may not generalize across environments, suggesting adaptive exploration durations might be better.

- **Bayesian vs. Frequentist Approaches in MAB** (4 points)

  Compare the intuition behind Bayesian approaches (such as Thompson Sampling) to frequentist methods (like UCB) in handling uncertainty. Under what conditions might the Bayesian approach yield superior practical performance, and how do the underlying assumptions about prior knowledge influence the explorationexploitation balance? Deep Dive: Explore the benefits of incorporating prior beliefs and the risk of bias if the prior is mis-specified, as well as how Bayesian updating naturally adjusts the exploration bonus as more data is collected.

  Answer:
  Bayesian methods like Thompson Sampling use prior distributions to model uncertainty, adapting exploration based on posterior updates. They can outperform frequentist methods when prior knowledge is informative or when adaptive exploration is beneficial. However, mis-specified priors can bias learning. Bayesian updates naturally adjust exploration over time by sampling from adjustable distributions, while frequentist methods use fixed bonuses or rules.

- **Impact of Skewed Reward Distributions** (3.75 points)

  In environments where one arm is significantly better (skewed probabilities), explain intuitively why agents like UCB or ExpFstAg might still struggle to consistently identify and exploit that arm. What role does variance play in these algorithms, and how might the skew exacerbate errors in reward estimation? Deep Dive: Discuss how the variability of rare but high rewards can mislead the agents estimates and cause prolonged exploration of suboptimal arms.

  Answer:
  When one arm is significantly better, Algorithms like UCB may continue exploring suboptimal arms due to conservative bonuses, especially if the best arm has high variance. This delays convergence and can cause extended regret. Skewed distributions also lead to biased Q-values in early stages, increasing the chance of misidentifying the best arm.

- **Designing for High-Dimensional, Sparse Contexts** (5 points)

In contextual bandits where the context is high-dimensional but only a few features are informative, what are the intuitive challenges that arise in using a linear model like LinUCB? How might techniques such as feature selection, regularization, or non-linear function approximation help, and what are the trade-offs involved? Deep Dive: Provide insights into the risks of overfitting versus underfitting, the increased variance in estimates from high-dimensional spaces, and the potential computational costs versus performance gains when moving from a simple linear model to a more complex one.

Answer:
In high-dimensional but sparse contexts, LinUCB may struggle due to overfitting, increased estimation variance, and high computational cost. Only a few features may be truly informative, so irrelevant ones dilute learning. Regularization can reduce overfitting, feature selection improves relevance, and non-linear methods (for example neural networks or kernels) offer better performance. However, these add computational overhead and may require more data to generalize well.